

ENGINEERING MANUAL

Unitree L1 Engineering Manual

Minimal Docker, ROS 2 Humble, RViz2, rosbag2, and OctoMap

Simplified OctoMap edition

Prepared 3 August 2026

Validated project baseline simplified OctoMap working-tree baseline

Audience: Robotics engineers, integrators, operators, and maintainers

DOCUMENT CONTROL

This document is generated from `engineering-manual.md`.

Commands and paths describe the repository state identified above.

Operate the powered LiDAR only after reading the safety section.

1. Purpose, scope, and evidence

This manual defines the simplified engineering baseline for reading a Unitree 4D LiDAR L1 over its serial adapter, publishing its native ROS 2 messages, viewing the point cloud in RViz2, building a live three-dimensional occupancy map while the sensor body is stationary, and recording the cloud and IMU with rosbag2. The complete ROS environment runs in Docker. The host supplies the physical device, graphical display, and persistent bag storage.

The design intentionally contains very little project code:

- One Docker image based on ROS 2 Humble.
- The official Unitree `unilidar_sdk` source at tag `v1.0.16`, commit

`1bd7d95d8ab7ce7a22058d2bb07e39fd62612aa6`, baked into the image.

- The vendor ROS 2 driver package, `unitree_lidar_ros2`.
- Two project-authored CMake packages: `l1_bringup` for raw sensor operation

and `l1_octomap_bringup` for the combined stationary OctoMap launch.

- The released ROS 2 Humble `octomap_server` package installed from apt; no

OctoMap source checkout or project mapping node.

- One persistent Compose service named `ros`, whose normal command is only

`sleep infinity`.

- Direct ROS 2 commands entered through `docker compose exec ros bash -l`.

There is no project monitor package, application framework, SLAM system, or hidden recording service. The optional monitor is exactly the command `ros2 topic hz /unilidar/cloud`, started with a ROS 2 launch `ExecuteProcess` action. Mapping uses the standard `octomap_server_node`.

1.1 Intended outcomes

The system is complete for the following engineering tasks:

- Open the selected serial device at the vendor driver's fixed 2,000,000 baud.
- Publish `sensor_msgs/msg/PointCloud2` and `sensor_msgs/msg/Imu`.
- Display the raw point cloud in its native LiDAR frame.
- Insert the live cloud into a 0.10 m OctoMap with a 15 m sensor range while

the L1 body remains stationary.

- Display occupied OctoMap voxels alongside the raw cloud in RViz2.
- Print the observed cloud arrival rate when requested.
- Record the two sensor topics into a rosbag2 directory.
- Inspect and replay a completed bag.
- Expose every persistent and task-specific process to the operator.

This baseline does not provide localization, odometry, SLAM, mobile mapping, navigation, obstacle avoidance, sensor fusion, calibration, or a robot frame tree. OctoMap estimates occupancy from already posed clouds; it does not estimate the LiDAR pose. Mobile use requires an external, time-aware pose and TF system and is outside this simplified stationary baseline.

1.2 Authority and validation language

The Dockerfile, Compose file, package manifest, CMake file, launch file, parameter file, and vendor source are authoritative. If this manual disagrees with them, stop and reconcile the difference before operating the sensor.

The following evidence was available during preparation:

Evidence	Result	Meaning
Vendor source audit	Passed	Package, parameters, topics, fields, and frames were read from source
Local Humble compile	Passed	Relevant vendor and bringup sources build in the local Jammy/Humble image
Docker graphics check	Passed	The local X11 and DRI path provided accelerated Intel rendering
Synthetic rosbag check	Passed	Humble recorded and reopened cloud and IMU test messages with sqlite3
Synthetic OctoMap check	Passed	Standard OctoMap consumed a cloud in <code>unilidar_lidar</code> and published non-empty map output without TF
Live L1 acceptance	Pending	No Unitree serial adapter was connected during the final audit

ENGINEERING NOTE

A successful build, an open RViz window, or a running driver process does not prove that the physical LiDAR is delivering valid data. Hardware acceptance requires messages from both topics and measured non-zero rates.

1.3 Compatibility statement

Unitree's documentation for this SDK names Ubuntu 20.04, ROS 2 Foxy, and PCL 1.10 as its verified environment. This project pins the same official `v1.0.16` source and has built it locally on Ubuntu 22.04, ROS 2 Humble, and PCL 1.12. That is project evidence, not a claim that Unitree officially supports Humble.

2. Safety and operating boundary

2.1 Powered hardware

SAFETY / WARNING

Secure the L1 before applying power. Its scanning mechanism can move. Keep hands, hair, loose clothing, cables, tools, and other objects outside the entire moving area.

Use the Unitree adapter and a correctly specified, separately connected power supply. Treat the USB connection as the serial data path, not as authorization to improvise sensor power. Disconnect power before changing wiring. Confirm polarity, voltage, connector, and current capability against the Unitree hardware documentation for the exact device revision.

Do not connect a TTL serial interface directly to RS-232 or to an arbitrary USB cable. Do not change the 2,000,000 baud value in the software without a controlled hardware test and supporting manufacturer information.

2.2 Host access

Expose only the identified serial character device to Docker. Identify it before launch:

```
lsusb
ls -l /dev/serial/by-id/ 2>/dev/null
ls -l /dev/ttyUSB* /dev/ttyACM* 2>/dev/null
```

If several serial adapters are present, use the stable `/dev/serial/by-id/...` link to determine the correct resolved tty. Recheck before each bench session; `/dev/ttyUSB0` numbering can change after a disconnect or reboot.

SAFETY / WARNING

Do not use `chmod 777`, `privileged: true`, or a global `xhost +`. Do not kill a process merely because it might own the port. First identify the device and inspect ownership with `stat` and `fuser`.

Useful read-only evidence is:

```
device=/dev/ttyUSB0
stat -Lc 'path=%n mode=%A uid=%u gid=%g' "$device"
fuser -v "$device"
```

The Compose configuration should map that one device to `/dev/unitree_lidar` in the container and grant the container user the device's numeric group. Broad host permission changes are unnecessary.

2.3 Process and data safety

Stop `ros2 launch` and `ros2 bag record` with `Ctrl-C`. In particular, `rosvbag2` must receive a clean interrupt so it can close its database and finish `metadata.yaml`. Do not unplug the sensor, stop the container, or power off the host while a bag is closing.

Bag output must use a new directory name. `rosvbag2` refuses an existing output directory; deleting or overwriting an old dataset is not part of routine operation.

3. Simplified architecture

3.1 Boundary and data flow

```
HOST
  Unitree adapter -> selected tty
  X11 socket and authorization
  DRI render device
  project bags directory
    |
    | explicit Compose device and bind mounts
    v
CONTAINER: service "ros"
  /sbin/docker-init -> sleep infinity
    |
    | docker compose exec ros bash -l
    v
  operator login shell
    |
    v
  ros2 launch ll_octomap_bringup unitree_ll_octomap.launch.py
    |
    +-> unitree_lidar_ros2_node
    |
    +-> /unilidar/cloud -> octomap_server_node -> occupied voxels
```

```

|         |                                     -> RViz2
|         |                                     -> ros2 topic hz, when enabled
|         |                                     -> rosbag2, when started separately
|         +--> /unilidar/imu -> rosbag2, when started separately
|
+--> ros2 topic hz /unilidar/cloud, optional process
+--> octomap_server_node
+--> octomap_rviz2, optional process

```

The image contains the software. The Compose service supplies a stable runtime boundary and access to host resources. The service itself does not start the driver, monitor, OctoMap, RViz, or recorder. Each task begins only when the operator enters a visible command.

3.2 Why the service stays idle

The `ros` service runs `Docker init` and `sleep infinity` so that:

- USB, X11, DRI, ROS domain, and storage mounts are configured once.
- Several explicit login shells can enter the same container.
- Launch and recording can run concurrently in separate terminals.
- Stopping a foreground task does not destroy the container.
- `docker compose ps` and `docker compose top` provide one clear audit point.

Docker Compose adds its standard small init process as PID 1 and runs `sleep infinity` as its child. The init process forwards stop signals and reaps exited children. Neither idle process reads the serial port or publishes ROS data. After `docker compose up -d`, a quiet ROS graph is expected until an operator starts a ROS command.

3.3 Package and file ownership

The installed ROS interface has four relevant package names:

Package	Provenance	Responsibility
<code>unitree_lidar_ros2</code>	Official Unitree repository	Serial parsing and message publication
<code>l1_bringup</code>	Project-authored CMake package	Parameters, launch composition, and RViz profile
<code>l1_octomap_bringup</code>	Project-authored CMake package	Combined OctoMap launch, parameters, and map RViz profile
<code>octomap_server</code>	ROS 2 Humble apt package	Standard occupancy insertion and map publication

The two project packages should remain small:

```

l1_bringup/
  CMakeLists.txt
  package.xml
  launch/
    unitree_l1.launch.py
  config/
    unitree_l1.yaml
    unitree_l1.rviz

l1_octomap_bringup/
  CMakeLists.txt
  package.xml
  launch/
    unitree_l1_octomap.launch.py
  config/
    octomap.yaml
    l1_octomap.rviz

```

There is no project Python node. Both CMake packages install data files. The mapping package includes the raw launch, adds the released OctoMap server, and owns one map-oriented RViz process. Recording is provided by the standard `ros2 bag` command installed in the image.

3.4 Exact runtime inventory

The normal process inventory is deliberately predictable:

Process or node	When present	Function
/sbin/docker-init	While service <code>ros</code> is up	PID 1; forwards signals and reaps children
<code>sleep infinity</code>	While service <code>ros</code> is up	Keeps the container available
Login bash	For each <code>docker compose exec session</code>	Operator command shell
<code>ros2 launch</code>	During a launched sensor session	Supervises launched children
/unitree_lidar_ros2_node	Always during sensor launch	Reads serial data and publishes both topics
<code>ros2 topic hz / unilidar/cloud</code>	Only with <code>monitor:=true</code>	Prints cloud arrival-rate statistics
/octomap_server	During the combined OctoMap launch	Converts the cloud into occupied and free voxels
/rviz2 or /octomap_rviz2	Only with <code>rviz:=true</code>	Displays the configured raw or mapped view
<code>ros2 bag record</code>	Only when separately requested	Subscribes and writes a bag

The raw launch omits `/octomap_server`. The combined launch always includes it. Never run both launches together because each launch starts its own copy of the serial driver.

`monitor:=true` does not create a stable project monitor node and does not publish `/diagnostics`. The ROS CLI rate command creates its own internal subscriber context; its implementation-generated node name is not a project API. The output appears in the `ros2 launch` terminal.

The recorder is also independent of the launch file. On Humble it is visible as the `rosbag2 recorder` process and recorder node while active. Stop it independently from the sensor launch.

Ordinary `ros2 node list` and `ros2 topic list` commands can start the standard ROS 2 CLI daemon. That daemon is tooling, not a project application. Inspection commands in this manual use `--no-daemon` where supported so that the audited process set stays small.

3.5 ROS domain and discovery

The Compose file does not set a custom domain, so this baseline uses the ROS 2 default `ROS_DOMAIN_ID=0`. All login shells in the same container therefore share the same DDS graph. A launch in one terminal and a recorder in a second terminal can discover one another.

Avoid mixing a host ROS installation, unrelated containers, or terminals with different domain IDs. If a future integration needs another domain, add it explicitly to Compose and recreate the service. If discovery is confusing, print the effective value in each shell:

```
printf 'ROS_DISTRO=%s ROS_DOMAIN_ID=%s\n' \
"$ROS_DISTRO" "${ROS_DOMAIN_ID:-0}"
```

4. Image build and reproducibility

4.1 Immutable vendor input

The image begins from the pinned `ros:humble-perception-jammy digest sha256:d89271d71fb7cefd5c39481b012172e4e8378835b8e6f23b19769b55a06dac35`. That official ROS image already supplies the PCL development stack required by the vendor driver. This project adds only RViz2, Mesa runtime support, and the released ROS 2 Humble `octomap_server` package instead of cloning OctoMap or reinstalling the complete PCL dependency tree. The project then checks out:

```
repository: https://github.com/unitreerobotics/unilidar_sdk.git
tag:        v1.0.16
commit:     1bd7d95d8ab7ce7a22058d2bb07e39fd62612aa6
```

The commit, not the moving tag name, is the reproducibility boundary. The Dockerfile must verify the checkout before compiling. Do not replace the commit with a branch such as `main`.

The vendor ROS 2 package links an architecture-specific prebuilt Unitree SDK archive. A build on a new CPU architecture is a separate compatibility test; a successful amd64 build is not evidence for another architecture.

4.2 Build and start

Run Docker operations from the repository root:

```
docker compose build
docker compose up -d
docker compose ps
```

Compose validates its required `DISPLAY` and `XAUTHORITY` substitutions even for a build, so use a terminal from the graphical desktop. The image build does not open the tty; the selected tty must exist when `up` creates the device-mapped service.

`docker compose build` creates the software image. `docker compose up -d` creates or starts the persistent `ros` service. `docker compose ps` must show that service running before `exec` is used.

Open the ROS environment directly:

```
docker compose exec ros bash -l
```

The login shell must source ROS 2 Humble and the image-built workspace. Confirm the sensor and mapping packages:

```
echo "$ROS_DISTRO"
ros2 pkg prefix unitree_lidar_ros2
ros2 pkg prefix l1_bringup
ros2 pkg prefix l1_octomap_bringup
ros2 pkg executables unitree_lidar_ros2
ros2 pkg executables octomap_server
```

EXPECTED RESULT

`ROS_DISTRO` is `humble`; all package-prefix commands succeed; the executable lists contain `unitree_lidar_ros2_node` and `octomap_server_node`.

4.3 Rebuild rule

Rebuild the image after changing the Dockerfile, pinned vendor commit, `package.xml`, `CMakeLists.txt`, launch file, parameter file, or RViz profile:

```
docker compose build
docker compose up -d --force-recreate
```

First stop active launch and recording commands cleanly. Recreating the idle service while applications are running terminates them and can damage an open bag.

5. ROS 2 sensor contract

5.1 Launch interfaces

The normal stationary mapping command is:

```
ros2 launch l1_octomap_bringup unitree_l1_octomap.launch.py \
  port:=/dev/unitree_lidar monitor:=true rviz:=true
```

It includes `l1_bringup`, forces the raw RViz child off, starts `octomap_server_node`, remaps its `cloud_in` input to `/unilidar/cloud`, and starts one map-oriented RViz process when requested. Its launch arguments are:

Argument	Default	Effect
<code>port</code>	<code>/dev/unitree_lidar</code>	Serial device visible inside the container
<code>monitor</code>	<code>true</code>	Start or omit the cloud-rate CLI process
<code>rviz</code>	<code>true</code>	Start or omit map-oriented RViz2
<code>resolution</code>	<code>0.10</code>	OctoMap voxel edge length in metres
<code>max_range</code>	<code>15.0</code>	Maximum sensor range inserted into OctoMap in metres

For a raw cloud without occupancy mapping, use this alternative:

```
ros2 launch l1_bringup unitree_l1.launch.py \
  port:=/dev/unitree_lidar monitor:=true rviz:=true
```

The raw launch arguments are:

Argument	Default	Effect
<code>port</code>	<code>/dev/unitree_lidar</code>	Serial device visible inside the container
<code>monitor</code>	<code>true</code>	Start or omit the cloud-rate CLI process
<code>rviz</code>	<code>true</code>	Start or omit RViz2

Use only `true` or `false` for the two Boolean arguments. With no ROS namespace, the YAML topic names resolve to `/unilidar/cloud` and `/unilidar/imu`.

Do not run the two launches together; each one starts a serial driver. To inspect both launch contracts without opening the device:

```
ros2 launch l1_bringup unitree_l1.launch.py --show-args
ros2 launch l1_octomap_bringup unitree_l1_octomap.launch.py --show-args
```

5.2 Driver parameters

The parameter file supplies the vendor driver:

Parameter	Baseline value	Engineering meaning
port	/dev/unitree_lidar	Container serial path
rotate_yaw_bias	0.0	Added yaw correction
range_scale	0.001	Vendor range scaling
range_bias	0.0	Added range correction
range_max	50.0	Maximum accepted range
range_min	0.0	Minimum accepted range
cloud_frame	unilidar_lidar	Cloud header frame
cloud_topic	unilidar/cloud	Cloud publisher name
cloud_scan_num	18	Vendor scans accumulated per cloud
imu_frame	unilidar_imu	IMU header frame
imu_topic	unilidar/imu	IMU publisher name

The `port` launch argument overrides only the YAML port. Topic, frame, range, and scan settings remain in the visible YAML file baked into the image. Changing that file requires an image rebuild.

The cloud topic is also named directly in the launch rate command and RViz profile. A topic change must update those three visible locations together; there is no topic-remapping launch argument in this simplified interface.

The serial baud is not a ROS parameter in the vendor node. Its source calls the SDK initializer with 2000000. Changing a launch argument cannot change that value.

5.3 Topics, types, and QoS

Topic	ROS type	Publisher contract
/unilidar/cloud	sensor_msgs/msg/PointCloud2	Reliable, Volatile, Keep Last, depth 10
/unilidar/imu	sensor_msgs/msg/Imu	Reliable, Volatile, Keep Last, depth 10

The QoS follows the vendor source's depth-10 `rclcpp` publishers in the Humble environment. Confirm the running graph rather than assuming:

```
ros2 topic info -v /unilidar/cloud --no-daemon
ros2 topic info -v /unilidar/imu --no-daemon
```

No `/diagnostics`, `/tf`, or `/tf_static` publisher is part of this minimal launch. Do not document or record those topics unless another explicitly started process supplies them.

5.4 OctoMap contract

The combined launch sets:

Parameter	Baseline	Meaning
frame_id	unilidar_lidar	Accumulate the map in the stationary sensor frame
resolution	0.10	Use 10 cm voxel edges
sensor_model.max_range	15.0	Ignore or truncate longer sensor rays for insertion
filter_ground_plane	false	Do not assume an unverified sensor-to-ground geometry
latch	true	Make map output available to late reliable subscribers

The principal mapping interfaces are:

Name	Type	Purpose
/unilidar/cloud	sensor_msgs/msg/PointCloud2	Input remapped to <code>cloud_in</code>
/occupied_cells_vis_array	visualization_msgs/msg/MarkerArray	Occupied voxels used by the RViz profile
/octomap_binary	octomap_msgs/msg/Octomap	Compact serialized occupancy tree
/octomap_full	octomap_msgs/msg/Octomap	Full serialized occupancy tree
/octomap_point_cloud_centers	sensor_msgs/msg/PointCloud2	Centers of occupied leaves
/projected_map	nav_msgs/msg/OccupancyGrid	Two-dimensional projection published by the server

No static TF publisher is needed in the supported stationary mode because the input cloud and OctoMap fixed frame have the same `unilidar_lidar` value. This does not compensate for physical sensor motion. If the housing translates or

rotates, successive scans are still inserted as though the sensor had not moved and the map becomes geometrically wrong.

Mobile mapping is a different integration: set the server frame to a stable frame such as `map` or `odom`, and provide the correct time-varying transform to `unilidar_lidar` at every cloud timestamp. This project deliberately does not pretend to provide that localization input.

5.5 Point-cloud schema

The vendor PCL point type registers these fields:

Field	Stored type	Meaning
<code>x</code>	<code>float32</code>	Cartesian X coordinate
<code>y</code>	<code>float32</code>	Cartesian Y coordinate
<code>z</code>	<code>float32</code>	Cartesian Z coordinate
<code>intensity</code>	<code>float32</code>	Return intensity
<code>ring</code>	<code>uint16</code>	Scan-ring identifier in the ROS cloud
<code>time</code>	<code>float32</code>	Point time relative to the cloud stamp

The SDK's internal ring value is wider, but the registered ROS PCL point type uses `uint16`. Downstream software must inspect the actual `PointCloud2` fields and must not assume a generic XYZI-only schema.

`cloud_scan_num=18` controls aggregation. It influences cloud size, update rate, and latency; the launch monitor reports only observed arrival rate and does not explain those tradeoffs.

5.6 IMU message

The driver fills:

- Header stamp and `unilidar_imu` frame ID.
- Orientation quaternion in `x, y, z, w` order.
- Three-axis angular velocity.
- Three-axis linear acceleration.

The vendor node does not fill the three covariance arrays, so they remain zero-initialized. A fusion system must not interpret those zeros as a validated noise model. Establish units, axis signs, calibration state, covariances, and time alignment experimentally before using the IMU for state estimation.

5.7 Frames and transforms

The official repository describes the LiDAR-frame origin at the center of the bottom mounting surface. Its positive X axis points opposite the outgoing cable, positive Y is 90 degrees counterclockwise from positive X, and positive Z points upward.

The IMU axes are parallel to the LiDAR axes. The vendor documentation gives the IMU origin in LiDAR coordinates as:

```
x = -0.007698 m
y = -0.014655 m
z = 0.006670 m
```

The driver sets frame IDs but publishes no transform between them. The project does not invent one. If a later system needs a TF edge, an engineer must verify the extrinsic against the hardware revision and publish it explicitly.

For stationary OctoMap, the map frame deliberately equals `unilidar_lidar`; TF2 therefore resolves the identity internally. This single-frame choice is a bench-mapping constraint, not a robot pose estimate.

5.8 Timestamp path

The vendor node converts the SDK's floating-point cloud and IMU stamps into ROS seconds and nanoseconds. It does not compare them with host time, check monotonicity, or report clock offset. `ros2 topic hz` measures message arrival intervals; it does not validate header timestamps.

For time-sensitive use, separately test:

- Stamps are non-zero and increasing.
- Cloud and IMU epochs match the intended clock.
- Header age is bounded.
- Restart and power-cycle behavior is understood.
- Bag replay preserves the recorded headers.

6. Operating procedure

6.1 Host preflight

1. Secure the LiDAR and complete manufacturer-approved wiring.
2. Connect the adapter and identify its stable serial path.
3. Confirm that no unexpected process owns the resolved tty.
4. Confirm the current graphical session has `DISPLAY` and a readable

`XAUTHORITY` when RViz will be used.

5. Confirm sufficient disk space for bag data.

```
printf 'DISPLAY=%s XAUTHORITY=%s\n' "$DISPLAY" "$XAUTHORITY"
df -h .
```

Set the host device input used by `compose.yaml` if it is not already in the environment. For example:

```
export LIDAR_DEVICE=/dev/ttyUSB0
export LIDAR_GID="$(stat -Lc '%g' "$LIDAR_DEVICE")"
```

Use the actual detected path. The container-facing launch path remains `/dev/unitree_lidar`.

6.2 Start the idle container

```
docker compose up -d
docker compose ps
docker compose top
```

At this point only the service's idle process and Docker process supervision should be present. The serial port should not yet be opened by the driver.

6.3 Start the live sensor and OctoMap

Open a login shell:

```
docker compose exec ros bash -l
```

For a first hardware check, start the combined mapping launch headless:

```
ros2 launch l1_octomap_bringup unitree_l1_octomap.launch.py \
  port:=/dev/unitree_lidar monitor:=true rviz:=false
```

The driver and rate monitor remain in the foreground under `ros2 launch`. Observe the driver output and wait for `ros2 topic hz` to print a measured average. Stop with `Ctrl-C` if the device cannot be opened or no data arrives.

After the headless checks pass, stop that launch and enable visualization:

```
ros2 launch l1_octomap_bringup unitree_l1_octomap.launch.py \
  port:=/dev/unitree_lidar monitor:=true rviz:=true
```

Keep the L1 housing fixed after mapping begins. The mechanism may scan internally, but the body and mounting surface must not translate or rotate. For raw cloud work without OctoMap, substitute the `l1_bringup` launch. Do not start two driver launches against the same serial port.

6.4 Inspect what is running

Keep the launch terminal visible. In a second host terminal:

```
docker compose top
docker compose exec ros bash -lc 'ros2 node list --no-daemon'
docker compose exec ros bash -lc 'ros2 topic list -t --no-daemon'
```

With the combined launch and `monitor:=true rviz:=true`, expect the driver, `/octomap_server`, and `/octomap_rviz2` nodes. The rate monitor is most reliably audited as the explicit `ros2 topic hz` process in `docker compose top`; its ROS CLI node name is not stable.

The expected sensor topics are:

```
/unilidar/cloud [sensor_msgs/msg/PointCloud2]
/unilidar/imu [sensor_msgs/msg/Imu]
/occupied_cells_vis_array [visualization_msgs/msg/MarkerArray]
/octomap_binary [octomap_msgs/msg/OctoMap]
```

Other ROS infrastructure topics can appear because ROS nodes publish their own parameter events or logs. They are not additional project sensor outputs.

6.5 RViz scope

The installed OctoMap RViz profile:

- Sets fixed frame to `unilidar_lidar`.
- Subscribes to `/unilidar/cloud`.
- Subscribes to `/occupied_cells_vis_array` with Transient Local durability.
- Draws the point cloud and occupied voxels in native sensor coordinates.
- Does not display or interpret the IMU.
- Does not create a robot model, localization estimate, or TF.

Because the cloud header and RViz fixed frame are the same, the raw cloud display needs no transform. If RViz says `No transform`, first verify that its fixed frame is exactly `unilidar_lidar` and that the received cloud has that frame ID.

An open but empty RViz window is not success. First require a live cloud. Then check that `/octomap_server` has one cloud subscription and that occupied map output is non-empty before changing graphics settings.

6.6 Stop

1. Press `Ctrl-C` in any active recorder terminal and wait for closure.
2. Press `Ctrl-C` in the launch terminal.
3. Exit the login shells.
4. Leave the idle service running for later work, or stop it explicitly.

```
docker compose stop
```

Use `docker compose down` only when the Compose container and network should be removed. Bag directories bind-mounted to the host remain persistent.

7. Recording and replay

7.1 Start a recording

Keep the live launch active. In a second terminal:

```
docker compose exec ros bash -l
mkdir -p /workspace/bags
ros2 bag record \
  -o /workspace/bags/11_20260730_130000 \
  /unilidar/cloud /unilidar/imu
```

Choose a new, descriptive output name for every run. The minimal dataset is exactly the two driver topics. There is no project `/diagnostics` or TF topic to add.

The Humble image uses `sqlite3` as its default `rosbag2` storage. During recording, monitor free space from another terminal:

```
df -h /workspace/bags
```

7.2 Close and verify

Press `Ctrl-C` once in the recorder terminal. Wait for `rosbag2` to finish before stopping the launch or container. Then run:

```
ros2 bag info /workspace/bags/11_20260730_130000
```

EXPECTED RESULT

The bag has `metadata.yaml`, one or more database files, both expected topic types, non-zero counts, and a plausible duration.

A directory existing on disk does not prove that it closed correctly. A bag that cannot be read by `ros2 bag info` is not accepted.

7.3 Replay a completed bag

While the Compose service is available, stop the live driver before replay so live and recorded topics are not mixed. The driver itself is not used during replay, but the current single-service Compose definition still requires the configured host `tty` when it creates the container. Leave the adapter and `tty` present; do not launch a second driver.

In one container shell, start RViz with the installed profile:

```
rviz2 -d \
  "$(ros2 pkg prefix --share 11_bringup)/config/unitree_l1.rviz"
```

In a second container shell:

```
ros2 bag play /workspace/bags/11_20260730_130000
```

The profile displays only the cloud. IMU messages are replayed but are not visualized by this configuration. Stop playback and RViz with `Ctrl-C`.

8. Verification and acceptance plan

8.1 Layered test strategy

Test one boundary at a time:

Level	Test boundary	Acceptance evidence
A	Image and packages	Build succeeds; package prefixes and executable exist
B	Container resources	Selected tty, X11, DRI, and bag path are accessible
C	Launch composition	<code>--show-args</code> matches the documented interface
D	ROS graph	Expected nodes, topic types, and QoS appear
E	Live hardware	Both topics deliver messages with valid frames and non-zero rates
F	Occupancy mapping	OctoMap subscribes once and publishes non-empty occupied output
G	Visualization	RViz renders changing cloud data and occupied voxels in <code>unilidar_lidar</code>
H	Recording	Cleanly closed bag reports both topics and non-zero counts
I	Replay	Recorded cloud is displayed without starting the driver

Do not skip directly to RViz. A blank display has too many possible causes to be a useful first test.

8.2 Software-only checks

The image can be checked without creating the device-mapped Compose service:

```
docker run --rm unitree-l1:humble-v1.0.16 bash -lc '
set -e
test "$ROS_DISTRO" = humble
ros2 pkg prefix unitree_lidar_ros2
ros2 pkg prefix l1_bringup
ros2 pkg prefix l1_octomap_bringup
ros2 pkg executables unitree_lidar_ros2
ros2 pkg executables octomap_server | grep -F "octomap_server octomap_server_node"
ros2 launch l1_bringup unitree_l1.launch.py --show-args
ros2 launch l1_octomap_bringup unitree_l1_octomap.launch.py --show-args
ros2 bag list storage
'
```

The software-only OctoMap acceptance publishes a synthetic PointCloud2 with frame `unilidar_lidar` to a standalone `octomap_server_node` configured from `octomap.yaml`. Require one `/unilidar/cloud` subscription, a non-zero width on `/octomap_point_cloud_centers`, and a non-empty binary `OcTree` on `/octomap_binary`. This test uses no TF publisher; matching input and map frames are sufficient for the stationary case.

Before graphical acceptance, verify that the declared container resources and RViz executable are present:

```
docker compose exec ros bash -lc '
printf "DISPLAY=%s XAUTHORITY=%s\n" "$DISPLAY" "$XAUTHORITY"
test -r "$XAUTHORITY"
test -d /tmp/.X11-unix
test -d /dev/dri
rviz2 --help >/dev/null
'
```

These checks prove resource and executable presence. The visual acceptance test is an RViz window that opens and renders the changing live cloud. The local audit observed accelerated Intel rendering; repeat visual acceptance after changing the host display session, GPU driver, or Compose mounts.

8.3 Live graph checks

With the LiDAR launch active, use a second login shell:

```
ros2 node list --no-daemon
ros2 topic list -t --no-daemon
ros2 param get /unitree_lidar_ros2_node port
ros2 topic info -v /unilidar/cloud --no-daemon
ros2 topic info -v /unilidar/imu --no-daemon
```

Require:

- `/unitree_lidar_ros2_node` is present.
- `/octomap_server` is present when the combined launch is used.
- The port parameter is `/dev/unitree_lidar`.
- Cloud and IMU types exactly match this manual.
- Each sensor topic has the vendor publisher.
- Offered QoS is Reliable, Volatile, Keep Last, depth 10.
- `/unilidar/cloud` has one OctoMap subscription in addition to any enabled monitor, RViz, recorder, or inspection subscribers.

Check that each topic delivers at least one message:

```
ros2 topic echo /unilidar/cloud \
  sensor_msgs/msg/PointCloud2 --once
ros2 topic echo /unilidar/imu \
  sensor_msgs/msg/Imu --once
```

The cloud output must include frame `unilidar_lidar` and fields `x`, `y`, `z`, `intensity`, `ring`, and `time`. The IMU output must include frame `unilidar_imu`.

Measure both rates for long enough to obtain stable averages:

```
ros2 topic hz /unilidar/cloud
ros2 topic hz /unilidar/imu
```

Stop each measurement with `Ctrl-C`. Do not impose an undocumented fixed rate. Record the measured average, minimum, maximum, standard deviation, test duration, `cloud_scan_num`, cable configuration, and sensor power state.

For stationary mapping acceptance, keep the sensor body fixed and require:

```
ros2 topic echo /octomap_point_cloud_centers \
  sensor_msgs/msg/PointCloud2 --once --field width
ros2 topic echo /octomap_binary \
  octomap_msgs/msg/Octomap --once --no-arr
```

The occupied-center width must become greater than zero and the binary message must identify an `OcTree` with a non-empty data field. Inspect `/occupied_cells_vis_array` in RViz only after these data-level checks pass.

8.4 Timestamp checks

Capture several headers rather than only one. Verify non-zero, increasing stamps and compare them with the ROS clock if the application depends on absolute time. Rate output alone cannot detect an incorrect epoch or stale header.

Power-cycle and relaunch as a separate test. Confirm whether stamps continue, reset, or jump, and make the downstream time policy explicit.

8.5 Hardware acceptance limitation

At preparation time, the host had no `/dev/unitree_lidar`, `/dev/ttyUSB*`, or `/dev/ttyACM*` device and no Unitree adapter was visible in the USB inventory. Consequently, this edition does not claim a live hardware pass, measured L1 rates, verified L1 timestamps, a real point cloud, or a live-hardware OctoMap. The software-only OctoMap path is tested separately with synthetic cloud data.

The live checks in this section are the required closure procedure when the hardware is connected. Record their results separately; do not silently upgrade this document's evidence statement.

9. Driver limitations and engineering implications

9.1 Initialization result

The vendor node calls the SDK initializer but does not check its return value. The ROS process can therefore remain alive even when the serial port failed to open.

SAFETY / WARNING

Never use process existence as the LiDAR health test. Require messages on both topics and measure their rates.

The simplest recovery is explicit: stop the launch, correct the device or permission problem, and launch again. The project does not add hidden retry or reconnection behavior.

9.2 Rate monitor scope

`ros2 topic hz /unilidar/cloud` reports observed arrival intervals. It does not verify:

- IMU arrival rate.
- Message type or point fields.
- Header timestamps or clock offset.
- Frame IDs.
- Point count, finite coordinates, or range validity.
- Serial errors, dropped bytes, temperature, or device health.
- Recording success.

It is a convenient live indicator, not diagnostics. Engineers who need machine-readable health must design that feature explicitly rather than calling this CLI output a safety monitor.

9.3 No transform publication

The driver publishes frame labels but no TF. Raw cloud RViz works because its fixed frame equals the cloud frame. Stationary OctoMap uses the same identity relationship by setting its map frame to `unilidar_lidar`. Any moving robot integration must add a verified extrinsic, localization source, and complete time-aware TF tree.

9.4 OctoMap is not localization

The map server updates occupancy from sensor rays but does not estimate where the sensor moved. Moving the L1 body under the default single-frame configuration corrupts the map without necessarily producing a process error. Treat a fixed mount as an operating requirement, not a convenience.

9.5 No covariance model

The IMU covariance arrays are not populated. Do not feed the message into a filter with assumed certainty. Establish noise and bias behavior over temperature and operating conditions, then publish an appropriate calibrated contract.

9.6 Vendor timestamp trust

The driver forwards the vendor stamp with no age, monotonicity, or synchronization check. Applications that combine cameras, odometry, or other sensors need a separate time-validation plan.

9.7 Fixed serial behavior

The node initializes the SDK at 2,000,000 baud and checks the parser on a 1-millisecond wall timer. The serial rate is not a launch option. CPU load, USB adapter behavior, and cloud aggregation should be measured on the target computer.

10. Troubleshooting

10.1 Work from the outside inward

Use this order:

```
physical power and cable
-> host USB enumeration
-> host tty identity and ownership
-> container /dev/unitree_lidar access
-> driver port parameter
-> ROS publisher and messages
-> OctoMap subscriber and occupied output
-> RViz or rosbag consumer
```

Changing RViz cannot fix a missing USB adapter. Changing file permissions cannot fix an unpowered sensor. Start at the first failed boundary.

10.2 Device absent on the host

Evidence:

```
lsusb
ls -l /dev/serial/by-id/ 2>/dev/null
dmesg --ctime | tail -n 40
```

Check sensor power, adapter, cable seating, and USB port. Compare USB inventory before and after connection. Do not assume every `/dev/ttyUSB0` is the L1.

10.3 Device absent or denied in the container

```
docker compose exec ros bash -lc '
ls -l /dev/unitree_lidar
stat -Lc "mode=%A uid=%u gid=%g" /dev/unitree_lidar
test -r /dev/unitree_lidar
test -w /dev/unitree_lidar
'
```

If the path is absent, inspect the Compose device mapping and recreate the idle service after correcting `LIDAR_DEVICE`. If permissions fail, compare the host tty GID with the supplementary groups inside the container. Do not solve the problem with world-writable permissions.

10.4 Driver runs but topics are silent

This is consistent with the ignored initialization return. Verify:

```
ros2 param get /unitree_lidar_ros2_node port
ros2 topic info -v /unilidar/cloud
ros2 topic info -v /unilidar/imu
```

Then stop the launch and check whether another process owns the host tty. Also confirm external power and the correct adapter. Restart only after the first failed boundary is corrected.

10.5 Monitor prints no average

The rate command prints an average only after receiving messages. Confirm the topic spelling and publisher first:

```
ros2 topic type /unilidar/cloud
ros2 topic info -v /unilidar/cloud
```

If the driver publishes under a customized topic, pass the same topic to a manually started rate command. In this simplified package, changing the baseline topic also requires coordinated edits to the YAML, launch rate command, and RViz profile followed by an image rebuild.

10.6 RViz does not open

Repeat the host display checks from Section 6.1. Inside the service:

```
printf 'DISPLAY=%s XAUTHORITY=%s\n' "$DISPLAY" "$XAUTHORITY"
test -r "$XAUTHORITY"
ls -l /tmp/.X11-unix /dev/dri
rviz2 --help >/dev/null
```

Recreate the Compose service from the current graphical session if it inherited stale display variables. Do not use unrestricted xhost access.

10.7 RViz opens but the cloud is absent

Require messages on `/unilidar/cloud`, type `sensor_msgs/msg/PointCloud2`, frame `unilidar_lidar`, the same RViz fixed frame, the correct display topic, and Reliable/Volatile subscription QoS.

If points exist but are not visible, reset the view and inspect point size, camera distance, and coordinate values. Do not add a fabricated TF.

10.8 OctoMap remains empty

Prove the raw input before inspecting visualization:

```
ros2 topic echo /unilidar/cloud --once --field header
ros2 node info /octomap_server
ros2 topic echo /octomap_point_cloud_centers --once --field width
```

The cloud frame must be `unilidar_lidar`, the OctoMap node must subscribe to `/unilidar/cloud`, and points must lie within the configured 15 m maximum range. Keep the sensor body stationary and allow several scans. Do not add a fabricated transform; the supported input and map frames already match.

10.9 Map RViz shows no occupied voxels

First require non-empty `/octomap_point_cloud_centers` or `/octomap_binary` data. Then confirm the RViz fixed frame is `unilidar_lidar`, the MarkerArray topic is `/occupied_cells_vis_array`, and the display is enabled. A healthy cloud display alone proves neither OctoMap insertion nor marker delivery.

10.10 Recorder reports zero messages

Confirm the recorder shell has the same `ROS_DOMAIN_ID` as the launch shell. Run `ros2 topic echo --once` before recording. Use exact absolute topic names. After stopping, require non-zero counts from `ros2 bag info`.

If the output directory already exists, choose a new name. Do not remove the old directory until its ownership and retention status are known.

If a bag does not reopen, retain the original directory unchanged for recovery work. Record a new, short, cleanly closed sample to distinguish a general storage failure from damage caused by an interrupted recorder.

10.11 Stale or duplicate processes

```
docker compose top
docker compose exec ros bash -lc 'ros2 node list --no-daemon'
```

Stop duplicate foreground launch or recording sessions with `Ctrl-C`. If only the idle service remains, it is safe to start a new task. Recreating the container is a last lifecycle step, not a substitute for understanding which process owned the device.

11. Maintenance and change control

Repeat the layered plan after changing vendor source, the base image, ROS, PCL, compiler, architecture, hardware, serial behavior, driver parameters, topics, frames, QoS, launch composition, Docker resources, RViz, or rosbag2. Record the image and source identifiers, host and adapter, parameters, measured rates, timestamp findings, and bag evidence.

Add a node or script only when a stated requirement cannot be met with a visible standard ROS 2 or Docker command. Every persistent process needs a documented purpose, lifecycle, interface, verification method, failure mode, and removal path. Convenience is not sufficient reason to hide a command.

Before session acceptance, account for secured stationary hardware, the correct adapter with narrow permissions and one driver, both message types and non-zero rates, tested frames, fields, and timestamps, non-empty OctoMap output, changing RViz data, a clean bag that passes `ros2 bag info`, and every task process. This closes the simple boundary:

serial input, two native messages, stationary occupancy mapping, optional rate and visualization, and explicit recording.